A trace semantics for the process calculus of ProVerif

Artie Khovanov with lots of assistance from Claude Fable

ProVerif is a tool for reasoning about cryptographic security in a symbolic model with a Dolev-Yao adversary. Although it uses resolution of Horn clauses to find attacks, it is backed theoretically by a process calculus, which is a variant of the applied pi calculus. The ground truth for what ProVerif code means is the formal semantics of its input language in Appendix B of the ProVerif manual [2], which takes precedence over the manual's informal chapters and over the tool itself: ProVerif's resolution procedure, and the encodings it applies to a model beforehand, approximate that semantics and are not consulted. Where the manual refers to a paper for a component, that paper is the ground truth for it: the adversary and the security properties are those of Blanchet's survey [1], to which the manual refers for them, extended to the manual's terms with effects. The attacker is the abstract one.

We formalise the ProVerif process calculus in Lean. Rather than copying the specification, we use a shallow translation in order to make security proofs simpler down the line. This means that we make use of existing Lean syntax and constructs where possible. In particular, some invariants are enforced only by what terms can arise from translating a valid ProVerif program. We also deviate from the calculus when sticking exactly to the specification would complicate the design.

We use a trace-based semantics to encode the possible outcomes of parallel processes communicating on a network. The attacker is modelled as a set of traces satisfying informational constraints. Our trace semantics are good enough to express secrecy and ProVerif's correspondence queries, nested and injective ones included. However, it is strictly less expressive than the formal operational semantics of ProVerif; in particular, we cannot express observational equivalence.

We give a mathematical account of the Lean development, and of the formal translation from terms in ProVerif's process calculus to our traces. Everything has been formalised in Lean.

Section 1 presents the trace semantics; Section 2 the calculus of ProVerif, its syntax and the semantics of the manual; and Section 3 the translation of the calculus into the trace semantics and the theorems relating the two. In the development these are the folders `TraceSemantics` and `Specification`, the translation being written twice: as the denotation in `Specification`, which the theorems are about, and as the program `Translation`, which turns a ProVerif model into Lean source over the trace semantics.

1 The trace semantics

1.1 Messages

Definition 1.1 (Signature, terms)**.** A *signature* is a family $\Sigma = (\Sigma_n)_{n\in\mathbb{N}}$ of sets; $f \in \Sigma_n$ is a constructor of arity n. The set $\mathcal{T} = \mathcal{T}(\Sigma)$ of *raw terms* is generated by

$$t ::= \nu_k \ (k \in \mathbb{N}) \ \mid \ f(t_1, \dots, t_n) \ (f \in \Sigma_n).$$

A *substitution* is a map $\sigma : \mathbb{N} \to \mathcal{T}$; $t\sigma$ replaces each ν_k in t by $\sigma(k)$.

Definition 1.2 (Theory)**.** A *theory* over Σ is a pair (E, Fr) with $E \subseteq \mathcal{T} \times \mathcal{T}$ and $\mathrm{Fr} \subseteq \bigcup_n \Sigma_n$ such that

(H) for every $(l, r) \in E$, both l and r are of the form $f(\dots)$ with $f \notin \mathrm{Fr}$.

The relation $\equiv_E$ is the least congruence on $\mathcal{T}$ containing $l\sigma \equiv_E r\sigma$ for all $(l, r) \in E$ and all substitutions σ. The set of *messages* is $\mathcal{M} = \mathcal{T}/\!\equiv_E$, with $[t]$ the class of t. Constructors act on $\mathcal{M}$ by $f([t_1], \dots, [t_n]) = [f(t_1, \dots, t_n)]$, and ν_k also denotes $[\nu_k]$.

A Σ-algebra A *satisfies* E if $l\sigma_A = r\sigma_A$ for every $(l, r) \in E$ and every valuation $\sigma_A : \mathbb{N} \to A$, where $t\sigma_A$ is the evaluation of t in A.

Proposition 1.3. *$\mathcal{M}$ satisfies E. For every Σ-algebra A satisfying E and every $v : \mathbb{N} \to A$ there is a unique homomorphism $\mathcal{M} \to A$ sending ν_k to $v(k)$.*

Definition 1.4 (View)**.** Let $\mathcal{V} = \mathbb{N} \uplus \{f(m_1, \dots, m_n) \mid f \in \mathrm{Fr} \cap \Sigma_n,\ m_i \in \mathcal{M}\} \uplus \{\bot\}$. Define $\mathrm{view}_0 : \mathcal{T} \to \mathcal{V}$ by

$$\mathrm{view}_0(\nu_k) = k, \qquad \mathrm{view}_0(f(t_1, \dots, t_n)) = \begin{cases} f([t_1], \dots, [t_n]) & f \in \mathrm{Fr}, \\ \bot & f \notin \mathrm{Fr}. \end{cases}$$

Proposition 1.5. *The following hold.*

1. *$s \equiv_E t$ implies $\mathrm{view}_0(s) = \mathrm{view}_0(t)$; hence view_0 induces $\mathrm{view} : \mathcal{M} \to \mathcal{V}$.*
2. *$\mathrm{view}(f(m_1, \dots, m_n)) = f(m_1, \dots, m_n)$ if $f \in \mathrm{Fr}$, and $\bot$ otherwise.*
3. *If $\mathrm{view}(m) = f(m_1, \dots, m_n)$ then $m = f(m_1, \dots, m_n)$.*
4. *$k \mapsto \nu_k$ is injective $\mathbb{N} \to \mathcal{M}$.*

Definition 1.6 (May-fail messages, lifting)**.** Let $\mathcal{M}_\bot = \mathcal{M} \uplus \{\bot\}$. For $g : \mathcal{M}^n \to \mathcal{M}_\bot$ let $g^\uparrow : \mathcal{M}_\bot^n \to \mathcal{M}_\bot$ be $g^\uparrow(\vec{u}) = g(\vec{u})$ if every $u_i \in \mathcal{M}$ and $\bot$ otherwise; $f^\uparrow$ for $f \in \Sigma_n$ is the lift of $f : \mathcal{M}^n \to \mathcal{M} \subseteq \mathcal{M}_\bot$. A *destructor* of arity n is a function $\mathcal{M}_\bot^n \to \mathcal{M}_\bot$.

Definition 1.7 (Data symbols, standard operations)**.** The *data symbols* are a tuple constructor tuple_n of each arity $n \in \mathbb{N}$ and the constants true and false. A signature *with data* is $\Sigma = \Sigma_0 \uplus \mathrm{Data}$ for a signature Σ_0 of *declared* symbols, and a theory (E, Fr) over Σ_0 is read over Σ with the data symbols free. We write $(m_1, \dots, m_n)$ for $\mathsf{tuple}_n(m_1, \dots, m_n)$. The *standard operations* Std are the lifts $\mathsf{tuple}_n^\uparrow$, the projections $\pi_i^n : \mathcal{M}_\bot \to \mathcal{M}_\bot$ with $\pi_i^n(u) = m_i$ if $u = (m_1, \dots, m_n)$ and $\bot$ otherwise, well defined by Proposition 1.5, and the constants true and false.

Definition 1.8 (Public operations, deduction)**.** Let $\mathrm{Pub} \subseteq \bigcup_n \Sigma_n$ and let Dt be a set of destructors. Set $\mathrm{Ops} = \{f^{\uparrow} \mid f \in \mathrm{Pub}\} \cup \mathrm{Dt} \cup \mathrm{Std}$. A set $S \subseteq \mathcal{M}$ is Ops-*closed* if for every $o \in \mathrm{Ops}$ of arity n and every $\vec{u} \in (S \uplus \{\bot\})^n$, $o(\vec{u}) \in \mathcal{M}$ implies $o(\vec{u}) \in S$. For $K \subseteq \mathcal{M}$,

$$\langle K \rangle = \bigcap \{S \supseteq K \mid S \text{ is Ops-closed}\}, \qquad K \vdash m \iff m \in \langle K \rangle.$$

Proposition 1.9. *$\langle\cdot\rangle$ is a closure operator on $\mathcal{P}(\mathcal{M})$, $\langle K \rangle$ is* Ops*-closed, and $K \vdash m_1, \ldots, K \vdash m_n$ imply $K \vdash f(m_1, \ldots, m_n)$ for $f \in \mathrm{Pub} \cap \Sigma_n$ or f a data symbol. In particular $\emptyset \vdash a$ for $a \in \mathrm{Pub} \cap \Sigma_0$.*

Remark 1.10. In the development the data symbols are added to a signature and a theory by withData, and the standard operations are the class StdOps on a type of messages, which the messages of a theory with data instantiate; Ops is the class Attacker, and Attacker.ofDecls builds it from Pub and Dt.

1.2 Traces and processes

Definition 1.11 (Actions, traces)**.**

$$\mathrm{Act} = \{\mathsf{out}(c,m),\ \mathsf{in}(c,m),\ \mathsf{comm}(c,m),\ \mathsf{new}(k),\ \mathsf{ev}(m) \mid c, m \in \mathcal{M},\ k \in \mathbb{N}\}.$$

A *trace* is a word $t \in \mathrm{Act}^*$; ε is the empty trace and $l \leq t$ means l is a prefix of t. The actions $\mathsf{out}(c,m)$ and $\mathsf{in}(c,m)$ are *unfinished*; $\mathrm{Closed}(t)$ holds if no action of t is unfinished. $\mathrm{nonces}(t) \in \mathbb{N}^*$ is the image of t under the monoid homomorphism with $\mathsf{new}(k) \mapsto k$ and every other action $\mapsto \varepsilon$. A *process* is a subset of Act^*; $\mathrm{Proc} = \mathcal{P}(\mathrm{Act}^*)$.

Definition 1.12 (Sequential constructors)**.** For $a \in \mathrm{Act}$, $P \in \mathrm{Proc}$, $F : \mathcal{M} \to \mathrm{Proc}$:

$$\begin{aligned} a \cdot P &= \{\varepsilon\} \cup \{a\,t \mid t \in P\}, & \mathbf{0} &= \{\varepsilon\}, \\ \mathsf{out}(c,m).P &= \mathsf{out}(c,m) \cdot P, & \mathsf{ev}(m).P &= \mathsf{ev}(m) \cdot P, \\ \mathsf{in}(c).F &= \textstyle\bigcup_{m \in \mathcal{M}} \mathsf{in}(c,m) \cdot F(m), & \mathsf{new}.F &= \textstyle\bigcup_{k \in \mathbb{N}} \mathsf{new}(k) \cdot \{t \in F(\nu_k) \mid k \notin \mathrm{nonces}(t)\}. \end{aligned}$$

Definition 1.13 (Parallel composition)**.** Let I be a set. For $s \in (I \times \mathrm{Act})^*$ and $i \in I$, $s\restriction i \in \mathrm{Act}^*$ is the word of actions of s tagged i, in order, and $\overline{s} \in \mathrm{Act}^*$ is s with tags removed. For $f : I \to \mathrm{Act}^*$, $\mathrm{Interleaves}(f,s)$ holds if $s\restriction i = f(i)$ for all i. The relation $\mathrm{comms} \subseteq (I \times \mathrm{Act})^* \times \mathrm{Act}^*$ is the least one such that

$$\begin{aligned} &\mathrm{comms}(\varepsilon, \varepsilon), \\ &\mathrm{comms}(s,t) \Rightarrow \mathrm{comms}\big((i,a)\,s,\ a\,t\big), \\ &\mathrm{comms}(s,t) \wedge i \neq j \Rightarrow \mathrm{comms}\big((i,\mathsf{out}(c,m))\,(j,\mathsf{in}(c,m))\,s,\ \mathsf{comm}(c,m)\,t\big). \end{aligned}$$

For $P : I \to \mathrm{Proc}$,

$$\begin{aligned} \mathrm{merge}_I(P) = \{t \mid \exists f, s.\ (\forall i.\ f(i) \in P(i)) \ \wedge\ &\mathrm{Interleaves}(f,s) \\ &\wedge\ \mathrm{comms}(s,t)\ \wedge\ \mathrm{nonces}(t) \text{ has no repeated letter}\}. \end{aligned}$$

$P \mid Q = \mathrm{merge}_{\{0,1\}}(0 \mapsto P,\ 1 \mapsto Q)$ and $!P = \mathrm{merge}_{\mathbb{N}}(k \mapsto P)$.

Proposition 1.14. *Let* $\mathrm{comms}(s,t)$.

1. $\mathrm{nonces}(t) = \mathrm{nonces}(\overline{s})$.
2. *If* a *occurs in* t *then* (i,a) *occurs in* s *for some* i*, or* $a = \mathsf{comm}(c,m)$ *and* $(i,\mathsf{out}(c,m))$, $(j,\mathsf{in}(c,m))$ *occur in* s *for some* $i \neq j$.
3. *If* $\mathrm{Closed}(t)$ *and* $(k,\mathsf{in}(c,m))$ *occurs in* s*, then* $(i,\mathsf{out}(c,m))$ *occurs in* s *for some* $i \neq k$.
4. $!P = P \mid !P$ *for every* $P \in \mathrm{Proc}$.

1.3 The attacker and the security properties

Definition 1.15 (Enemy)**.** The processes $\mathrm{Enemy}(K) \in \mathrm{Proc}$, for $K \subseteq \mathcal{M}$, are the pointwise least such that

1. $\varepsilon \in \mathrm{Enemy}(K)$;
2. if $K \vdash c$ and $e \in \mathrm{Enemy}(K \cup \{m\})$ then $\mathsf{in}(c,m)\, e \in \mathrm{Enemy}(K)$, for every $m \in \mathcal{M}$;
3. if $K \vdash c$, $K \vdash m$ and $e \in \mathrm{Enemy}(K)$ then $\mathsf{out}(c,m)\, e \in \mathrm{Enemy}(K)$;
4. if $K \vdash c$, $K \vdash m$ and $e \in \mathrm{Enemy}(K)$ then $\mathsf{comm}(c,m)\, e \in \mathrm{Enemy}(K)$, an output to itself;
5. if $k \notin \mathrm{nonces}(e)$ and $e \in \mathrm{Enemy}(K \cup \{\nu_k\})$ then $\mathsf{new}(k)\, e \in \mathrm{Enemy}(K)$.

Definition 1.16 (Attacks, knowledge)**.** For $P \in \mathrm{Proc}$,

$$\mathrm{Attacks}(P) = \{t \in P \mid \mathrm{Enemy}(\emptyset) \ : \ \mathrm{Closed}(t)\}.$$

For $K \subseteq \mathcal{M}$, $\mathrm{known}(K,\cdot) : \mathrm{Act}^* \to \mathcal{P}(\mathcal{M})$ is defined by

$$\begin{aligned} \mathrm{known}(K,\varepsilon) &= K, \\ \mathrm{known}(K,a\,t) &= \mathrm{known}\big(\langle K \cup \{m \mid a = \mathsf{comm}(c,m),\ c \in K\}\rangle,\, t\big). \end{aligned}$$

Proposition 1.17 (Prefix-closure)**.** $\mathrm{Enemy}(K)$ *and every process built from the constructors of Definitions 1.12 and 1.13 are prefix-closed. If* P *is prefix-closed, so is* $\mathrm{Attacks}(P)$.

Definition 1.18 (Facts, secrecy, correspondence)**.** A *fact* is $\mathsf{event}(e)$ or $\mathsf{knows}(m)$ with $e, m \in \mathcal{M}$; Fact is the set of facts. A fact *holds* after $t \in \mathrm{Act}^*$, written $t \models f$, if

$$t \models \mathsf{event}(e) \iff \mathsf{ev}(e) \in t, \qquad t \models \mathsf{knows}(m) \iff m \in \mathrm{known}(\langle\emptyset\rangle, t).$$

For a set X, $F, H : (X \to \mathcal{M}) \to \mathrm{Fact}$ and $s \in \mathcal{M}$,

$$\begin{aligned} \mathrm{Corr}(P,F,H) \iff{}& \forall \sigma : X \to \mathcal{M}.\ \forall t \in \mathrm{Attacks}(P). \\ & t \models F(\sigma) \Rightarrow \exists \sigma' : X \to \mathcal{M}.\ F(\sigma') = F(\sigma) \wedge t \models H(\sigma'), \\ \mathrm{Secret}(P,s) \iff{}& \forall t \in \mathrm{Attacks}(P).\ t \not\models \mathsf{knows}(s). \end{aligned}$$

By Proposition 1.17, for prefix-closed P these are properties of every prefix of every attack: $\mathrm{Corr}(P,F,H)$ says that some $H(\sigma')$ holds by the time $F(\sigma)$ does.

Definition 1.19 (Query facts)**.** A *query fact* is $\mathsf{event}(e)$, $\mathsf{knows}(m)$, $\mathsf{learns}(m)$, $\mathsf{mess}(c,m)$, $m = n$ or $m \neq n$ with $e, c, m, n \in \mathcal{M}$, or f before e with f a query fact and $e \in \mathcal{M}$; QFact is the set of query facts, and Fact $\subseteq$ QFact. A query fact *holds at step* i of $t = a_1 \cdots a_n \in \mathrm{Act}^*$, for $0 \leq i \leq n$, written $t, i \models f$, if

$$\begin{aligned}
t, i \models \mathsf{event}(e) &\iff i \geq 1 \wedge a_i = \mathsf{ev}(e),\\
t, i \models \mathsf{mess}(c,m) &\iff i \geq 1 \wedge a_i = \mathsf{comm}(c,m),\\
t, i \models \mathsf{knows}(m) &\iff m \in \mathrm{known}(\langle\emptyset\rangle, a_1 \cdots a_i),\\
t, i \models \mathsf{learns}(m) &\iff t, i \models \mathsf{knows}(m) \wedge (i = 0 \vee t, i-1 \not\models \mathsf{knows}(m)),\\
t, i \models (m = n) &\iff m = n, \qquad t, i \models (m \neq n) \iff m \neq n,\\
t, i \models f \ \mathsf{before}\ e &\iff t, i \models f \wedge \mathsf{ev}(e) \notin \{a_1, \ldots, a_i\}.
\end{aligned}$$

A fact $f \in$ Fact holds after t exactly when it holds at some step $i \leq n$ of t.

Definition 1.20 (Hypotheses, queries, witnesses)**.** For a set Φ, the *hypotheses* over Φ are generated by

$$H ::= F \mid F^{\mathsf{inj}} \mid H \mathbin{\&\&} H \mid H \mid\mid H \mid \mathsf{false} \mid (F \Longrightarrow H) \mid (F^{\mathsf{inj}} \Longrightarrow H) \qquad (F \in \Phi),$$

where F^{inj} is F marked *injective*, and a *query* over Φ is $F_1 \mathbin{\&\&} \ldots \mathbin{\&\&} F_n \Longrightarrow H$ with $n \geq 1$ elements of Φ before the arrow, each possibly marked injective, and a hypothesis H after it. Let $\models \subseteq \mathbb{N} \times \Phi$ be a relation, $i \models F$ read "F holds at step i". A *witness* of H *by step* $d \in \mathbb{N}$ is: for F or F^{inj}, a step $j \leq d$ with $j \models F$; for $H_1 \mathbin{\&\&} H_2$, a witness of each by d; for $H_1 \mid\mid H_2$, a witness of one of them by d; for $(F \Longrightarrow H')$ or $(F^{\mathsf{inj}} \Longrightarrow H')$, a step $j \leq d$ with $j \models F$ together with a witness of H' by j; and false has none. A witness w assigns a step $w(o)$ to each occurrence o of an injective fact of H that it passes through.

Let S be a set of *valuations* and q a query $F_1 \mathbin{\&\&} \ldots \mathbin{\&\&} F_n \Longrightarrow H$ over $S \to \Phi$, whose facts are functions $F(\sigma)$ of a valuation; $H(\sigma)$ is H with each fact applied to σ. For $N \in \mathbb{N}$, a valuation σ and steps $\vec{\imath} = (i_1, \ldots, i_n)$ are *valid* if $i_k \leq N$ and $i_k \models F_k(\sigma)$ for each k. The query q is *satisfied in* N *steps* if there is a choice $w(\sigma, \vec{\imath})$, for every valid $(\sigma, \vec{\imath})$, of a witness of $H(\sigma')$ by step $\max \vec{\imath}$ for some σ' with $F_k(\sigma') = F_k(\sigma)$ for $k = 1, \ldots, n$, such that for every occurrence o of an injective fact of H and all valid $(\sigma_1, \vec{\imath}_1)$ and $(\sigma_2, \vec{\imath}_2)$ with $w(\sigma_1, \vec{\imath}_1)(o) = w(\sigma_2, \vec{\imath}_2)(o)$, the steps of $\vec{\imath}_1$ and $\vec{\imath}_2$ at the injective facts among $F_1, \ldots, F_n$ agree. For $P \in$ Proc and a query q over $(X \to \mathcal{M}) \to$ QFact,

$$\mathrm{Satisfies}(P, q) \iff \forall t \in \mathrm{Attacks}(P).\ q \text{ is satisfied in } |t| \text{ steps, with } i \models f \text{ when } t, i \models f.$$

Proposition 1.21 (Basic queries)**.** *Let* $F, H : (X \to \mathcal{M}) \to$ Fact. *Then* $\mathrm{Satisfies}(P, F \Longrightarrow \mathsf{false})$ *is* $\forall \sigma.\ \forall t \in \mathrm{Attacks}(P).\ t \not\models F(\sigma)$*, which for* $F = \sigma \mapsto \mathsf{knows}(s)$ *is* $\mathrm{Secret}(P, s)$*; and if* P *is prefix-closed,* $\mathrm{Satisfies}(P, F \Longrightarrow H)$ *is* $\mathrm{Corr}(P, F, H)$.

2 The calculus of ProVerif

This section gives the syntax of the fragment of ProVerif's input language that the development covers and restates its semantics from the ProVerif manual (Appendix B). The manual describes the attacker and the properties its queries test informally (§3.2, §4.3.1) and refers to the survey

for their definitions (Chapter 7), so the adversary and the basic correspondences are the survey's (§3.1, §3.3), stated here for the manual's semantics; secrecy, the attacker and message facts and the nested and injective correspondences follow the manual's descriptions (§4.3.1), secrecy deviating from the survey's as Definition 2.8 and its remark say.

2.1 Syntax

Definition 2.1 (Source syntax)**.** Fix a set of variables $x, y, \ldots$ and a set of names $a, b, c, \ldots$ containing the free names $\mathcal{N}_{\text{pub}} \uplus \mathcal{N}_{\text{priv}}$. Terms, expressions and processes are generated by

$$\begin{aligned} M, N &::= x \mid a \mid f(M_1, \ldots, M_n), \\ D &::= M \mid h(D_1, \ldots, D_n) \mid \mathsf{fail} \mid \mathsf{new}\ a; D \mid \mathsf{if}\ D\ \mathsf{then}\ D_1\ \mathsf{else}\ D_2 \\ &\quad \mid \mathsf{let}\ x = D\ \mathsf{in}\ D_1\ \mathsf{else}\ D_2 \mid \mathsf{event}\ ev(D_1, \ldots, D_n); D', \\ P, Q &::= \mathbf{0} \mid \mathsf{out}(D, D'); P \mid \mathsf{in}(D, x); P \mid P \mid Q \mid !P \mid \mathsf{new}\ a; P \\ &\quad \mid \mathsf{let}\ x = D\ \mathsf{in}\ P\ \mathsf{else}\ Q \mid \mathsf{if}\ D\ \mathsf{then}\ P\ \mathsf{else}\ Q \mid \mathsf{event}\ ev(D_1, \ldots, D_n); P, \end{aligned}$$

where f ranges over constructors, h over constructors and destructors and ev over event symbols. A term is an expression, and a *may-fail term* is $U ::= M \mid \mathsf{fail}$. An *event* e is $ev(M_1, \ldots, M_n)$ with ev an event symbol of arity n and $M_1, \ldots, M_n$ terms; it is not a term, and $=_E$ and substitution extend to events componentwise. Types are erased, and with them the [typeConverter] constructors, which are the identity when types are ignored (§4.1.2). These are the manual's terms, process terms and processes (Appendix A, Figures A.1 and A.8) without tables, predicates, phases, synchronisation and natural numbers; patterns other than variables are abbreviations, Definition 2.4. The manual's $x \leftarrow D; D'$ and $x \leftarrow D; P$ abbreviate $\mathsf{let}\ x = D\ \mathsf{in}\ D'\ \mathsf{else}\ \mathsf{fail}$ and $\mathsf{let}\ x = D\ \mathsf{in}\ P\ \mathsf{else}\ \mathbf{0}$, its $a \stackrel{R}{\leftarrow} t$ abbreviates $\mathsf{new}\ a$, and an omitted else is $\mathsf{else}\ \mathsf{fail}$ in an expression and $\mathsf{else}\ \mathbf{0}$ in a process. An application $g(D_1, \ldots, D_n)$ of a function macro $\mathsf{letfun}\ g(x_1, \ldots, x_n) = D$ (§4.2.3) abbreviates $\mathsf{let}\ x_1 = D_1\ \mathsf{in}\ \ldots\ \mathsf{let}\ x_n = D_n\ \mathsf{in}\ D$ with every else omitted, except that the let of a parameter x_i declared $\mathsf{or}\ \mathsf{fail}$ has as its else branch the rest, $\mathsf{let}\ x_{i+1} = D_{i+1}\ \mathsf{in}\ \ldots\ D$, with fail for x_i: the failure of the argument is passed to the macro instead of failing the application (§4.2.3). A call $p(D_1, \ldots, D_n)$ of a process macro $\mathsf{let}\ p(x_1, \ldots, x_n) = P$ (§3.1.3, §4.2.4) likewise abbreviates $\mathsf{let}\ x_1 = D_1\ \mathsf{in}\ \ldots\ \mathsf{let}\ x_n = D_n\ \mathsf{in}\ P$ with every else omitted, the else branch of a parameter declared $\mathsf{or}\ \mathsf{fail}$ being again the rest with fail for it (§4.2.4).

Definition 2.2 (Rewrite rules)**.** Fix a set of *may-fail variables* u, disjoint from the variables of Definition 2.1, which are the *ordinary variables*. The may-fail terms of a rule are $U ::= M \mid u \mid \mathsf{fail}$ with M a term without names: the may-fail terms of Definition 2.1 with may-fail variables besides. Each destructor g of arity n carries a finite ordered list $\mathrm{def}(g)$ of *rewrite rules* $g(U_1, \ldots, U_n) \to U$ between may-fail terms, the variables of U occurring in $U_1, \ldots, U_n$, ending with $g(\vec{u}) \to \mathsf{fail}$ for distinct may-fail variables $\vec{u}$. A *substitution* maps ordinary variables to terms and may-fail variables to may-fail terms. The manual writes $\mathrm{def}(g)$ as a set of rules each with a side condition that excludes the rules before it (Appendix B); ProVerif generates that form from the ordered list, and the two define the same rewriting. A destructor declared by reduc with its rules separated by ';' (§3.1.1) has them as a set rather than a list, which the manual requires to be deterministic:

(D) two of its rules that apply to the same arguments give the same result.

Its $\mathrm{def}(g)$ is the list in the order written, which under (D) rewrites as the set does.

Definition 2.3 (Model)**.** A *model* consists of finite disjoint sets $\mathcal{N}_{\mathrm{pub}}$ and $\mathcal{N}_{\mathrm{priv}}$ of free names; a finite set of *declared* function symbols, each with an arity, each a constructor or a destructor, and each public or private, the destructors with their rules $\mathrm{def}(g)$ as in Definition 2.2; a finite set of *event symbols*, each with an arity; an *equational theory* E, a finite set of equations $M = N$ between terms over the constructors without names; and a closed process P_0 with $\mathrm{fn}(P_0) \subseteq \mathcal{N}_{\mathrm{pub}} \uplus \mathcal{N}_{\mathrm{priv}}$. The constructors of the calculus are the declared ones and the *data symbols*, a tuple constructor $(\cdot, \ldots, \cdot)$ of each arity and the constants true and false; its destructors are the declared ones and the *built-in* ones, the projections $i\mathrm{th}_n$, each with the single rule $i\mathrm{th}_n((x_1, \ldots, x_n)) \to x_i$, and the boolean destructors equal, not, and and or with the rules

$$\begin{array}{llll} \mathsf{equal}(x, x) \to \mathsf{true}, & \mathsf{equal}(x, y) \to \mathsf{false}, & \mathsf{not}(\mathsf{true}) \to \mathsf{false}, & \mathsf{not}(x) \to \mathsf{true}, \\ \mathsf{and}(\mathsf{true}, u) \to u, & \mathsf{and}(x, u) \to \mathsf{false}, & \mathsf{or}(\mathsf{true}, u) \to \mathsf{true}, & \mathsf{or}(x, u) \to u, \end{array}$$

u a may-fail variable, so that $\mathsf{and}(\mathsf{false}, \mathsf{fail})$ rewrites to false; $M = N$, $M \neq N$, $M \mathbin{\&\&} N$, $M \mid\mid N$ and $\mathsf{not}(M)$ abbreviate the expressions $\mathsf{equal}(M, N)$, $\mathsf{not}(\mathsf{equal}(M, N))$, $\mathsf{and}(M, N)$, $\mathsf{or}(M, N)$ and $\mathsf{not}(M)$. The rules of every destructor are over the constructors of the calculus. The data symbols and the built-in destructors are public. These are the survey's tuples, projections and booleans (§2.1, (2.5)) and the manual's infix operators (Appendix B), built in as ProVerif has them. A declared constructor f of arity n may be a *data constructor* (§4.1.2): it is public, and for each i its *projection* f_i, a declared destructor with the single rule $f_i(f(x_1, \ldots, x_n)) \to x_i$, is public too.

Definition 2.4 (Patterns)**.** A *pattern* is $T ::= x \mid _ \mid {=}D \mid (T_1, \ldots, T_n) \mid f(T_1, \ldots, T_n)$ with f a data constructor (§3.1.4, §4.1.2). Its *variables* are the x in it, its *tests* the D of its ${=}D$, and its *leaves* the occurrences of x, $_$ and ${=}D$, all left to right; the *path* of a leaf is the composite of the projections reading it, $i\mathrm{th}_n$ for the i-th component of a tuple and f_i for that of f. The manual's $\mathsf{let}\ T = D\ \mathsf{in}\ P\ \mathsf{else}\ Q$ and $\mathsf{in}(D, T); P$ (Figure A.8) abbreviate processes of Definition 2.1, and $\mathsf{let}\ T = D\ \mathsf{in}\ D_1\ \mathsf{else}\ D_2$ (Figure A.1) an expression, the same way with fail for Q: for the tests $D_1, \ldots, D_k$, the leaves with paths $t_1, \ldots, t_L$, of which the i_1-th, …, i_k-th are the tests and the j_1-th, … the variables, and fresh variables x, z, $a_1, \ldots, a_L$,

$$\begin{array}{rcl} \mathsf{let}\ T = D\ \mathsf{in}\ P\ \mathsf{else}\ Q & \text{is} & \mathsf{let}\ x = D\ \mathsf{in}\ (\mathsf{let}\ z = (D_1, \ldots, D_k)\ \mathsf{in}\ L\ \mathsf{else}\ Q) \\ & & \quad \mathsf{else}\ (\mathsf{let}\ z = (D_1, \ldots, D_k)\ \mathsf{in}\ Q\ \mathsf{else}\ Q), \\ & L = & \mathsf{let}\ a_1 = t_1(x)\ \mathsf{in}\ \ldots\ \mathsf{let}\ a_L = t_L(x)\ \mathsf{in}\ C\ \mathsf{else}\ Q\ \ldots\ \mathsf{else}\ Q, \\ & C = & \mathsf{if}\ a_{i_1} = 1\mathrm{th}_k(z)\ \mathsf{then}\ \ldots\ \mathsf{if}\ a_{i_k} = k\mathrm{th}_k(z)\ \mathsf{then}\ P\{a_{j_1}/x_1, \ldots\} \\ & & \quad \mathsf{else}\ Q\ \ldots\ \mathsf{else}\ Q, \end{array}$$

and $\mathsf{in}(D, T); P$ is $\mathsf{in}(D, x); \mathsf{let}\ T = x\ \mathsf{in}\ P\ \mathsf{else}\ \mathbf{0}$. This has the transitions of the manual's rules for patterns (Appendix B, Figures B.1 and B.2): the term is evaluated first, then every test, whether or not the term failed; and $\mathrm{matches}(T, U)$ is defined exactly when no test failed, every projection succeeds, which for the free data symbols is that U has the pattern's shape, and every test equals its component modulo $=_E$, its substitution being the binding of the variable leaves.

Definition 2.5 (Queries)**.** A *query fact* is $\mathsf{event}(e)$, $\mathsf{inj\text{-}event}(e)$, $\mathsf{attacker}(M)$ or $\mathsf{mess}(N, M)$ with e an event and M, N terms, all over constructors, free names and a finite set X of variables; it is an *event fact*, *injective* when written $\mathsf{inj\text{-}event}$, an *attacker fact* or a *message fact* accordingly

(§4.3.1). Besides these, F before e with F a query fact and e an event is a query fact of the development and not of ProVerif: the encoding of phases in Section 3.1 produces it. A *hypothesis* is generated by

$$H ::= F \mid M = N \mid M <> N \mid H \,\&\&\, H \mid H \mid\mid H \mid \mathsf{false} \mid (F \Longrightarrow H),$$

with M, N terms over constructors, free names and X, && binding tighter than ||, the last form being a *nested correspondence*; an equality or a disequality is a fact after the arrow only (§4.3.1). A *query* is query $F_1 \,\&\&\, \ldots \,\&\&\, F_n \Longrightarrow H$ with $n \geq 1$ query facts before the arrow and a hypothesis after it, all over X; query $F_1 \,\&\&\, \ldots \,\&\&\, F_n$ abbreviates it with $H = \mathsf{false}$; a fact before the arrow counts as injective when H contains an injective fact, whether or not it is written inj-event; and query attacker(M) with M without variables is the secrecy query of §3.2. The manual states the injectivity rule for an event before the arrow (§4.3.1) and defines injectivity by executions of events only; the development extends it to the other facts, the executions of a message fact being its communications and those of an attacker fact the steps at which the attacker acquires the term (Definition 2.8). A query, a hypothesis or a fact may be prefixed by bindings let $x = M$ in with M such a term (§6.4), which substitute M for x; and a term may refer to a bound name, new n, denoting any name created at the restriction new n of the process, each occurrence a name of its own unless a let binds one variable to it (§6.4). The references to bound names, the manual's table facts, its qualification of a fact by a phase and its query secret x are compiled away before translation (Section 3.1); its inequalities between naturals, its is_nat and its time variables $F@t$ are outside the fragment.

2.2 Semantics

Names are atoms. The equational theory E of the model gives $=_E$, the closure of E under reflexivity, symmetry, transitivity, application of function symbols and substitution. The manual states its semantics for the full input language, in Figures B.1 and B.2, and reads it modulo $=_E$: a step on a term $M' =_E M$ counts as a step on M. Here its rules are restricted to the fragment of Definition 2.1, and the tables, phases and synchronisation parameters that the fragment does not use are dropped from its configurations; the communication rule, which in the manual leaves the input to match its pattern in a further step, matches the variable at once.

Rewriting. A closed destructor application $g(U_1, \ldots, U_n)$ *rewrites to* U, written $g(\vec{U}) \to U$, if $\mathrm{def}(g) = (g(U'_{i,1}, \ldots, U'_{i,n}) \to U'_i)_{i \leq k}$, j is least such that some substitution σ has $\sigma U'_{j,i} =_E U_i$ for $i = 1, \ldots, n$, and $U = \sigma U'_j$. The final rule of $\mathrm{def}(g)$ makes j exist, so every closed destructor application rewrites.

Expressions. A *term configuration* is $\mathcal{N}, D$ with $\mathcal{N}$ a finite set of names disjoint from $\mathcal{N}_{\mathrm{pub}}$ and D a closed expression with $\mathrm{fn}(D) \subseteq \mathcal{N}_{\mathrm{pub}} \cup \mathcal{N}$. Writing ℓ for a label that is either empty or

event e for a closed event e, the transitions $\mathcal{N}, D \xrightarrow{\ell}_t \mathcal{N}', D'$ are the least relation closed under

$$\begin{aligned}
\mathcal{N}, f(U_1, \ldots, U_n) &\to_t \mathcal{N}, \mathsf{fail} && \text{if some } U_i = \mathsf{fail} \\
\mathcal{N}, g(U_1, \ldots, U_n) &\to_t \mathcal{N}, U && \text{if } g(U_1, \ldots, U_n) \to U \\
\mathcal{N}, \mathsf{new}\ a; D &\to_t \mathcal{N} \cup \{a'\}, D\{a'/a\} && \text{if } a' \notin \mathcal{N}_{\mathrm{pub}} \cup \mathcal{N} \\
\mathcal{N}, \mathsf{if}\ M\ \mathsf{then}\ D_1\ \mathsf{else}\ D_2 &\to_t \mathcal{N}, D_1 && \text{if } M =_E \mathsf{true} \\
\mathcal{N}, \mathsf{if}\ M\ \mathsf{then}\ D_1\ \mathsf{else}\ D_2 &\to_t \mathcal{N}, D_2 && \text{if } M \neq_E \mathsf{true} \\
\mathcal{N}, \mathsf{if\ fail\ then}\ D_1\ \mathsf{else}\ D_2 &\to_t \mathcal{N}, \mathsf{fail} \\
\mathcal{N}, \mathsf{let}\ x = M\ \mathsf{in}\ D_1\ \mathsf{else}\ D_2 &\to_t \mathcal{N}, D_1\{M/x\} \\
\mathcal{N}, \mathsf{let}\ x = \mathsf{fail}\ \mathsf{in}\ D_1\ \mathsf{else}\ D_2 &\to_t \mathcal{N}, D_2 \\
\mathcal{N}, \mathsf{event}\ ev(M_1, \ldots, M_n); D &\xrightarrow{\mathsf{event}\ ev(M_1,\ldots,M_n)}_t \mathcal{N}, D \\
\mathcal{N}, \mathsf{event}\ ev(U_1, \ldots, U_n); D &\to_t \mathcal{N}, \mathsf{fail} && \text{if some } U_i = \mathsf{fail} \\
\mathcal{N}, C[D] &\xrightarrow{\ell}_t \mathcal{N}', C[D'] && \text{if } \mathcal{N}, D \xrightarrow{\ell}_t \mathcal{N}', D',\ C \in \mathcal{C}_t,
\end{aligned}$$

where f is a constructor, g a destructor and $\mathcal{C}_t$ is the set of contexts

$$\begin{gathered}
h(U_1, \ldots, U_{i-1}, [\,], D_{i+1}, \ldots, D_n), \qquad \mathsf{if}\ [\,]\ \mathsf{then}\ D_1\ \mathsf{else}\ D_2, \\
\mathsf{let}\ x = [\,]\ \mathsf{in}\ D_1\ \mathsf{else}\ D_2, \qquad \mathsf{event}\ ev(U_1, \ldots, U_{i-1}, [\,], D_{i+1}, \ldots, D_n); D.
\end{gathered}$$

So the arguments of an application or an event are evaluated from left to right, each to a may-fail term before the next, and the evaluation goes on after an argument has failed; a may-fail term has no transition.

Processes. A *configuration* is $\mathcal{N}, \mathcal{P}$ with $\mathcal{N}$ as above and $\mathcal{P}$ a finite multiset of closed processes with $\mathrm{fn}(\mathcal{P}) \subseteq \mathcal{N}_{\mathrm{pub}} \cup \mathcal{N}$. The transitions $\mathcal{N}, \mathcal{P} \xrightarrow{\ell} \mathcal{N}', \mathcal{P}'$ are the least relation closed under

$$\begin{aligned}
\mathcal{N}, \mathcal{P} \cup \{\mathbf{0}\} &\to \mathcal{N}, \mathcal{P} \\
\mathcal{N}, \mathcal{P} \cup \{P \mid Q\} &\to \mathcal{N}, \mathcal{P} \cup \{P, Q\} \\
\mathcal{N}, \mathcal{P} \cup \{!P\} &\to \mathcal{N}, \mathcal{P} \cup \{P, !P\} \\
\mathcal{N}, \mathcal{P} \cup \{\mathsf{new}\ a; P\} &\to \mathcal{N} \cup \{a'\}, \mathcal{P} \cup \{P\{a'/a\}\} && \text{if } a' \notin \mathcal{N}_{\mathrm{pub}} \cup \mathcal{N} \\
\mathcal{N}, \mathcal{P} \cup \{\mathsf{out}(N, M); Q,\ \mathsf{in}(N', x); P\} &\to \mathcal{N}, \mathcal{P} \cup \{Q,\ P\{M/x\}\} && \text{if } N =_E N' \\
\mathcal{N}, \mathcal{P} \cup \{\mathsf{if}\ M\ \mathsf{then}\ P\ \mathsf{else}\ Q\} &\to \mathcal{N}, \mathcal{P} \cup \{P\} && \text{if } M =_E \mathsf{true} \\
\mathcal{N}, \mathcal{P} \cup \{\mathsf{if}\ M\ \mathsf{then}\ P\ \mathsf{else}\ Q\} &\to \mathcal{N}, \mathcal{P} \cup \{Q\} && \text{if } M \neq_E \mathsf{true} \\
\mathcal{N}, \mathcal{P} \cup \{\mathsf{let}\ x = M\ \mathsf{in}\ P\ \mathsf{else}\ Q\} &\to \mathcal{N}, \mathcal{P} \cup \{P\{M/x\}\} \\
\mathcal{N}, \mathcal{P} \cup \{\mathsf{let}\ x = \mathsf{fail}\ \mathsf{in}\ P\ \mathsf{else}\ Q\} &\to \mathcal{N}, \mathcal{P} \cup \{Q\} \\
\mathcal{N}, \mathcal{P} \cup \{\mathsf{event}\ ev(M_1, \ldots, M_n); P\} &\xrightarrow{\mathsf{event}\ ev(M_1,\ldots,M_n)} \mathcal{N}, \mathcal{P} \cup \{P\} \\
\mathcal{N}, \mathcal{P} \cup \{C[D]\} &\xrightarrow{\ell} \mathcal{N}', \mathcal{P} \cup \{C[D']\} && \text{if } \mathcal{N}, D \xrightarrow{\ell}_t \mathcal{N}', D',\ C \in \mathcal{C}_p,
\end{aligned}$$

where $\mathcal{C}_p$ is the set of contexts

$$\begin{gathered}
\mathsf{out}([\,], D); P, \qquad \mathsf{out}(U, [\,]); P, \qquad \mathsf{in}([\,], x); P, \qquad \mathsf{if}\ [\,]\ \mathsf{then}\ P\ \mathsf{else}\ Q, \\
\mathsf{let}\ x = [\,]\ \mathsf{in}\ P\ \mathsf{else}\ Q, \qquad \mathsf{event}\ ev(U_1, \ldots, U_{i-1}, [\,], D_{i+1}, \ldots, D_n); P.
\end{gathered}$$

A process whose expressions have become may-fail terms with fail where a term is required, as out(fail, M); P, in(fail, x); P, if fail then P else Q or event ev(fail); P, has no transition. A *trace* is a finite transition sequence $\mathcal{N}_0, \mathcal{P}_0 \xrightarrow{\ell_1} \ldots \xrightarrow{\ell_n} \mathcal{N}_n, \mathcal{P}_n$, written $\mathcal{N}_0, \mathcal{P}_0 \rightarrow^* \mathcal{N}_n, \mathcal{P}_n$.

Definition 2.6 (Survey, Definitions 3.1, 3.4 and 3.5)**.** Let P_0 be the closed process of the model.

1. A closed process Q is an $\mathcal{N}_{\text{pub}}$-*adversary* if $\text{fn}(Q) \subseteq \mathcal{N}_{\text{pub}}$, every function symbol of Q is public, and Q contains no event.
2. A trace *executes* event e if some step of it is labelled event e' with $e' =_E e$.
3. P_0 *satisfies* event$(e) \rightsquigarrow \bigvee_{j=1}^{m} \bigwedge_{k=1}^{l_j}$ event(e_{jk}) *against* $\mathcal{N}_{\text{pub}}$-*adversaries*, for events e and e_{jk} with variables, if, for every $\mathcal{N}_{\text{pub}}$-adversary Q, every trace Tr from $\mathcal{N}_{\text{priv}}, \{P_0, Q\}$ and every substitution σ: if Tr executes event σe then there are σ' and j with $\sigma' e =_E \sigma e$ such that Tr executes event $\sigma' e_{jk}$ for every $k \leq l_j$.

The survey states these for its own calculus. Here the traces are those of the manual's semantics above; an adversary may use new, if and let inside its terms, and fail, which the input language lacks but which if false then true with its else omitted evaluates to (§4.1.4), as the survey's clause (Rfail) gives the adversary fail outright (§3.2.2); and the set of private names, which the survey chooses with the model, is $\mathcal{N}_{\text{priv}}$.

For an $\mathcal{N}_{\text{pub}}$-adversary Q write $\mathcal{C}_Q$ for the configuration $\mathcal{N}_{\text{priv}}, \{P_0, Q\}$.

Definition 2.7 (Deduction, knowledge)**.** The attacker *derives* a closed term M from a set S of closed terms, written $S \Vdash M$, if this follows from: $S \Vdash M$ when $M =_E M'$ for some $M' \in S$; $S \Vdash f(M_1, \ldots, M_n)$ when f is a public constructor, a data symbol among them, and $S \Vdash M_1, \ldots, S \Vdash M_n$; and $S \Vdash M$ when g is a public destructor, a built-in one among them, $g(U_1, \ldots, U_n) \rightarrow M$ and $S \Vdash U_i$ for every U_i that is a term. This is what the survey's adversary computes, applying the public function symbols to what it has (§3.1). The attacker's *knowledge* $K(\text{Tr})$ after a trace Tr from $\mathcal{C}_Q$ is the set of closed terms collected along it: none at first, the public names being public nullary symbols, and, at each communication step with the output out$(N, M'); Q'$, the term M' when $K \Vdash N$ for the knowledge K collected so far, since the attacker intercepts every message on a channel it has, whoever received it. The attacker *knows* M after Tr if $K(\text{Tr}) \Vdash M$.

Definition 2.8 (Satisfaction, secrecy)**.** Let Tr be a trace $\mathcal{C}_Q = \mathcal{C}_0 \xrightarrow{\ell_1} \mathcal{C}_1 \cdots \xrightarrow{\ell_N} \mathcal{C}_N$ and $0 \leq i \leq N$. Tr *satisfies* a closed query fact *at step* i as follows: event(e) and inj-event(e) if $i \geq 1$ and ℓ_i is event e' with $e' =_E e$; mess(N, M) if $i \geq 1$ and the i-th step is a communication of M' on N' with $N' =_E N$ and $M' =_E M$, the manual's "M is sent on channel N" (§4.3.1); attacker(M) if the attacker knows M after $\mathcal{C}_0 \rightarrow \cdots \rightarrow \mathcal{C}_i$, which is how the manual reads an attacker fact on either side of a correspondence (§4.3.1); an injective attacker(M), before the arrow, if it satisfies attacker(M) at step i and, unless $i = 0$, not at step $i - 1$, the step at which the attacker acquires M; $M = N$ at every step if $M =_E N$, and $M <> N$ at every step if not; and F before e if it satisfies F at step i and no ℓ_j with $j \leq i$ is event e' with $e' =_E e$. Tr *satisfies* a closed query fact if it does at some step; for an event fact that is Definition 2.6(2). P_0 *preserves the secrecy of* the closed term M if no trace from $\mathcal{C}_Q$, for any $\mathcal{N}_{\text{pub}}$-adversary Q, satisfies attacker(M): the reachability of the fact, which is what ProVerif decides for query attacker(M).

Remark 2.9. Secrecy deviates from the survey's Definitions 3.2 and 3.3, to which the manual

refers for it. There a trace *outputs M publicly* if some communication step of it has the output $\mathsf{out}(N, M'); Q$ with $N \in \mathcal{N}_{\mathrm{pub}}$ and $M' =_E M$, and P_0 preserves the secrecy of M from $\mathcal{N}_{\mathrm{pub}}$ if no trace against an $\mathcal{N}_{\mathrm{pub}}$-adversary outputs M publicly. The attacker can output on a public name whatever it knows, so when $\mathcal{N}_{\mathrm{pub}} \neq \emptyset$ the two definitions agree. But $\mathcal{N}_{\mathrm{pub}}$ holds the names declared free or channel and not the public constants, which are nullary constructors (manual, Appendix B), and a model may declare its public channel as a constant, as the Noise Explorer models do with const pub: such a model has $\mathcal{N}_{\mathrm{pub}} = \emptyset$, so by Definition 3.3 it preserves the secrecy of every term, whereas ProVerif finds the attacks on it. The strict reading is inconsistent with the tool, and Definition 2.8 states what the tool implements.

Definition 2.10 (Correspondences)**.** Let q be the query $F_1 \,\&\&\, \ldots \,\&\&\, F_n \Longrightarrow H$ over X, let Tr be a trace of N steps from $\mathcal{C}_Q$, and let substitutions σ send X to closed terms. A *witness* of a closed hypothesis H' *by step* $d \leq N$ in Tr is: for a fact F, a step $j \leq d$ at which Tr satisfies F; for $H_1 \,\&\&\, H_2$, a witness of each by d; for $H_1 \,||\, H_2$, a witness of one of them by d; for $(F \Longrightarrow H'')$, a step $j \leq d$ at which Tr satisfies F together with a witness of H'' by j; and false has none. A witness w assigns a step $w(o)$ to each occurrence o of an injective fact of H that it passes through. A substitution σ and steps $\vec{\imath} = (i_1, \ldots, i_n)$ with $i_k \leq N$ are *valid* if Tr satisfies σF_k at step i_k for each k. Tr *satisfies* q if there is a choice $w(\sigma, \vec{\imath})$, for every valid $(\sigma, \vec{\imath})$, of a witness of $\sigma' H$ by step $\max \vec{\imath}$ for some σ' with $\sigma' F_k =_E \sigma F_k$ for $k = 1, \ldots, n$, such that for every occurrence o of an injective fact of H and all valid $(\sigma_1, \vec{\imath}_1)$ and $(\sigma_2, \vec{\imath}_2)$ with $w(\sigma_1, \vec{\imath}_1)(o) = w(\sigma_2, \vec{\imath}_2)(o)$, the steps of $\vec{\imath}_1$ and $\vec{\imath}_2$ at the injective facts among $F_1, \ldots, F_n$ agree. P_0 *satisfies* q *against* $\mathcal{N}_{\mathrm{pub}}$*-adversaries* if every trace from $\mathcal{C}_Q$, for every $\mathcal{N}_{\mathrm{pub}}$-adversary Q, satisfies q.

This reads the manual's descriptions (§4.3.1) on the traces. A basic correspondence $F_1 \,\&\&\, \ldots \,\&\&\, F_n \Longrightarrow H$ holds when, whenever the facts before the arrow hold, H holds, which ProVerif proves by stopping the execution just after the last of them, so the witness is by their last step; a nested correspondence $(F \Longrightarrow H'')$ inside H asks that H'' be witnessed by the step of F, the manual's "the events of H'' have been executed before F, or at the same time"; and an injective fact in H asks for distinct executions of it for distinct tuples of executions of the injective facts before the arrow. For one event fact before the arrow and H a disjunction of conjunctions of event facts, none injective, this is Definition 2.6(3), the traces from $\mathcal{C}_Q$ being closed under prefixes. With $H = \mathsf{false}$, P_0 satisfies q exactly when no trace satisfies $\sigma F_1, \ldots, \sigma F_n$ for any σ, the facts being "not simultaneously reachable".

3 The translation

The calculus is translated into the trace semantics: a destructor to a function on may-fail messages, an expression to a computation, a process to a set of traces and a query to the property it denotes. Section 3.3 relates the translation to the semantics of Section 2.

Throughout, fix a model (Definition 2.3) and instantiate the trace semantics of Section 1 from it: the signature Σ_0 has the declared constructors, the event symbols and the free names $\mathcal{N}_{\mathrm{pub}} \uplus \mathcal{N}_{\mathrm{priv}}$ as nullary symbols, so that $\Sigma = \Sigma_0 \uplus \mathrm{Data}$ of Definition 1.7 has the data symbols besides and an event $ev(\vec{m})$ is a message; E is the equational theory of the model; Fr is the set of symbols of Σ_0 at the root of no equation, which contains the event symbols; Pub is the set of public declared constructors together with $\mathcal{N}_{\mathrm{pub}}$, so that the event symbols are private; and Dt is the set of the functions of Definition 3.1 of the public declared destructors, so that the attacker has the standard operations besides. A projection then denotes the standard projection π_i^n, and a

boolean destructor is not among Ops, which changes nothing, each of its results being true, false or an argument. The instantiation requires

(R1) no equation of E has a variable as a side, so that (E, Fr) satisfies (H);

(R2) in every rule of every destructor, the terms among the arguments have all their constructors in Fr.

These conditions are a restriction on the models translated, and a deviation from the manual, whose rewriting modulo $=_E$ needs neither: the translator rejects an equation with a variable as one side, such as the survey's (2.8) or the block cipher of the manual's §4.2.2, and a destructor whose rules mention a symbol at the root of an equation.

3.1 Compilations before the translation

Five features outside the fragment are compiled away on a model before it is translated, in this order, each to what the manual or this section gives as its encoding. The compilations belong to the translator and are not formalised: the specification of Section 2 and the theorems of Section 3.3 concern the compiled model.

The secrecy of a bound name or variable. The manual's query secret x (§4.3.1) tests whether the attacker can compute a value bound to x, a name bound by new x or a variable bound by an input or a let; a binder inside a term is not allowed. The translator compiles it to the reachability query it is: an event symbol x_{s} of arity one is declared, event $x_{\mathsf{s}}(x)$ is inserted after each binder of x in the process and its macros, and the query becomes query x; $\mathsf{event}(x_{\mathsf{s}}(x))$ && $\mathsf{attacker}(x)$.

Bound names in queries. The parser makes each reference new n in a query (Definition 2.5) a variable v of the query, the occurrences that a let binds to one variable sharing it. The translator then compiles the reference as it compiles secrecy: an event symbol n_{r} of arity one is declared, event $n_{\mathsf{r}}(n)$ is inserted after each new n of the process and its macros, and the fact $\mathsf{event}(n_{\mathsf{r}}(v))$ is added before the arrow when v occurs there, and conjoined to each fact after the arrow that mentions v otherwise, so that v is any name created at that restriction. A nested correspondence whose own fact mentions v is not translated.

Tables. Tables (§4.1.5) are outside the fragment. The translator compiles them to private channels before translating, by the manual's encoding (§6.7.3): a table d is a private free name, and

$$\begin{aligned} \mathsf{insert}\ d(M_1,\ldots,M_n);P \quad &\text{is} \quad \mathsf{let}\ r=(M_1,\ldots,M_n)\ \mathsf{in}\ ((!\,\mathsf{out}(d,r)) \mid P),\\ \mathsf{get}\ d(T_1,\ldots,T_n)\ \mathsf{suchthat}\ M\ \mathsf{in}\ P \quad &\text{is} \quad \mathsf{in}(d,(T_1,\ldots,T_n));\mathsf{if}\ M\ \mathsf{then}\ P. \end{aligned}$$

A get with an else branch, which runs when no record matches, has no such encoding and is not translated. The encoding departs from the manual's semantics of tables (Appendix B) in one way: an input may take a record that its pattern rejects and stop, a trace that executes no event and adds nothing to the attacker's knowledge. When a query has a fact $\mathsf{table}(d(M_1,\ldots,M_n))$, an event symbol d_{i} of arity one is declared, event $d_{\mathsf{i}}(r)$ is inserted before the supply of each insert into d, and the fact becomes $\mathsf{event}(d_{\mathsf{i}}((M_1,\ldots,M_n)))$, the fact of the insertion's step. Tables as such are left for later.

Synchronisation. A synchronisation $\mathsf{sync}\ t\ [tag]; P$ (§4.1.7) waits until every synchronisation of level t in the process has been reached, of which there are finitely many since none may be under a replication, and the levels are passed in increasing order. The compiled model declares, for each level t present, the private free names $\mathsf{barrier}_t$ and resume_t, rewrites $\mathsf{sync}\ t\ [tag]; P$ to $\mathsf{out}(\mathsf{barrier}_t, \mathsf{barrier}_t); \mathsf{in}(\mathsf{resume}_t, _); P$, and runs beside the process the coordinator of the least level,

$$C_t = \underbrace{\mathsf{in}(\mathsf{barrier}_t, _); \cdots ; \mathsf{in}(\mathsf{barrier}_t, _)}_{k_t}; \big(\underbrace{\mathsf{out}(\mathsf{resume}_t, \mathsf{resume}_t) \mid \cdots \mid \mathsf{out}(\mathsf{resume}_t, \mathsf{resume}_t)}_{k_t} \mid C_{t'} \big),$$

where t' is the next level present, C of no level is $\mathbf{0}$, and k_t is the number of distinct tags of level t, an untagged synchronisation counting as a tag of its own: a level completes once each of its tags has reported, the processes waiting at it are then released, and the next level is attended to only afterwards. A model with both synchronisations and phases is refused, as the manual forbids the combination.

Phases. Phases (§4.1.6) are outside the fragment: $\mathsf{phase}\ t; P$ runs P in phase t, the attacker advances the global clock from each phase to the next when it chooses, every process that has not reached the new phase is then discarded, and processes communicate only within a phase. The translator compiles them away, after the tables, into the fragment. Let n be the greatest phase of the model. The compiled model declares the public constants $\mathsf{ph}_0, \ldots, \mathsf{ph}_n$, the public free names $\mathsf{clock}_1, \ldots, \mathsf{clock}_n$, the private free names $\mathsf{go}_1, \ldots, \mathsf{go}_n$ and $\mathsf{tok}_0, \ldots, \mathsf{tok}_n$ and the events $\mathsf{tick}_1, \ldots, \mathsf{tick}_n$, and runs beside its process the clock $\mathsf{out}(\mathsf{tok}_0, \mathsf{ph}_0) \mid C_1$, where

$$C_t = \mathsf{in}(\mathsf{clock}_t, _); \mathsf{in}(\mathsf{tok}_{t-1}, _); \mathsf{event}\ \mathsf{tick}_t; \\ \big((!\,\mathsf{out}(\mathsf{go}_t, \mathsf{ph}_t)) \mid \mathsf{out}(\mathsf{tok}_t, \mathsf{ph}_t) \mid C_{t+1} \big), \qquad C_{n+1} = \mathbf{0}.$$

A process under phase t is rewritten by

$$\mathsf{out}(N, M); P \mapsto \mathsf{out}((N, \mathsf{ph}_t), M); P, \qquad \mathsf{in}(N, T); P \mapsto \mathsf{in}((N, \mathsf{ph}_t), T); \mathsf{in}(\mathsf{tok}_t, v); \mathsf{out}(\mathsf{tok}_t, v); P,$$
$$\mathsf{phase}\ s; P \mapsto \mathsf{in}(\mathsf{go}_s, _); P \quad (s > t), \qquad \mathsf{phase}\ s; P \mapsto \mathbf{0} \quad (s \le t),$$

with v a fresh variable, the P under $\mathsf{phase}\ s$ then being rewritten under phase s; the main process is under phase 0 and a process macro under the phase of its calls, which must agree; and the supply $!\,\mathsf{out}(d, r)$ of a compiled table under phase t becomes $!\,\mathsf{out}((d, \mathsf{ph}_t), r) \mid \cdots \mid !\,\mathsf{out}((d, \mathsf{ph}_n), r)$, a table keeping its elements across phases. In a query, $\mathsf{attacker}(M)\ \mathsf{phase}\ s$ and $\mathsf{table}(\ldots)\ \mathsf{phase}\ s$ with $s < n$ become $F\ \mathsf{before}\ \mathsf{tick}_{s+1}$ for the compiled fact F, the attacker's knowledge growing along a trace and the attacker advancing the clock at will, and with $s = n$ or without the qualifier, "in any phase", F itself; $\mathsf{mess}(N, M)\ \mathsf{phase}\ s$ becomes $\mathsf{mess}((N, \mathsf{ph}_s), M)$, and $\mathsf{mess}(N, M)$ without a qualifier is of the last phase (§4.3.1).

The encoding is exact, in this sense: every trace of the compiled process is, up to moving the actions of the processes that a transition discards before that transition, a trace of the manual's semantics with phases, and conversely. The attacker advances the clock by sending on clock_t, which it has. A process communicates on (N, ph_t), so only with processes of its phase and with the attacker, which knows ph_t; a process under $\mathsf{phase}\ s$ waits for go_s, which the clock supplies without bound once phase s has started. The attacker may address any phase at any time, but what it sends tagged ph_t after the transition to phase $t+1$ reaches only a process of phase t left

behind, which the manual discards: such a process may still receive once, but the token tok_t it then needs was taken away for good by the transition, so it never acts on what it received, and the trace is a prefix of one in which it received nothing. Its other actions after the transition, outputs, events and the creation of names, depend on nothing that happened after it and could have been performed before it. The claim is not formalised.

3.2 Destructors, expressions, processes and queries

Definition 3.1 (Destructors from rewrite rules)**.** A *valuation* θ sends ordinary variables to $\mathcal{M}$ and may-fail variables to $\mathcal{M}_\perp$, and gives a may-fail term the value $U\theta \in \mathcal{M}_\perp$: the evaluation $M\theta \in \mathcal{M}$ of a term, $\theta(u)$, and $\perp$ for fail. A destructor g with $\mathrm{def}(g) = (g(U_{i,1}, \ldots, U_{i,n}) \to U_{i,0})_{i \le k}$ denotes the function

$$g(\vec{u}) = U_{i,0}\theta \quad \text{for } i \text{ least such that some } \theta \text{ has } U_{i,j}\theta = u_j \text{ for } j = 1, \ldots, n,$$

which the final rule of $\mathrm{def}(g)$ makes total. By (R2) and Proposition 1.5 at most one θ on the variables of the i-th rule has $U_{i,j}\theta = u_j$ for all j, so g is well defined. A term among the $U_{i,j}$ thus accepts only messages, fail only $\perp$, and a may-fail variable any u_j, which is the survey's two kinds of substitution. When no rule but the final one contains fail or a may-fail variable, every other rule requires messages, so $g = g_0^{\uparrow}$ for the $g_0 : \mathcal{M}^n \to \mathcal{M}_\perp$ that those rules define on messages.

Definition 3.2 (Translation)**.** An *environment* ρ maps variables and names to $\mathcal{M}$; ρ_0 maps each free name a to the constant $a \in \mathcal{M}$. Write $\rho M \in \mathcal{M}$ for the homomorphic extension. An expression is translated to a *computation* $[\![D]\!]\rho : (\mathcal{M}_\perp \to \mathrm{Proc}) \to \mathrm{Proc}$, the process it becomes when its result is passed to a continuation κ:

$$\begin{aligned}
[\![M]\!]\rho\,\kappa &= \kappa(\rho M),\\
[\![\mathsf{fail}]\!]\rho\,\kappa &= \kappa(\perp),\\
[\![h(D_1, \ldots, D_n)]\!]\rho\,\kappa &= [\![D_1]\!]\rho\,\big(u_1 \mapsto [\![D_2]\!]\rho\,\big(u_2 \mapsto \ldots\\
&\qquad \ldots [\![D_n]\!]\rho\,\big(u_n \mapsto \kappa(h(u_1, \ldots, u_n))\big)\ldots\big)\big),\\
[\![\mathsf{new}\ a; D]\!]\rho\,\kappa &= \mathsf{new}.\big(m \mapsto [\![D]\!]\rho[a \mapsto m]\,\kappa\big),\\
[\![\mathsf{if}\ D\ \mathsf{then}\ D_1\ \mathsf{else}\ D_2]\!]\rho\,\kappa &= [\![D]\!]\rho\,\Big(u \mapsto \begin{cases} [\![D_1]\!]\rho\,\kappa & u = \mathsf{true},\\ [\![D_2]\!]\rho\,\kappa & u \in \mathcal{M} \setminus \{\mathsf{true}\},\\ \kappa(\perp) & u = \perp, \end{cases}\Big)\\
[\![\mathsf{let}\ x = D\ \mathsf{in}\ D_1\ \mathsf{else}\ D_2]\!]\rho\,\kappa &= [\![D]\!]\rho\,\Big(u \mapsto \begin{cases} [\![D_1]\!]\rho[x \mapsto u]\,\kappa & u \in \mathcal{M},\\ [\![D_2]\!]\rho\,\kappa & u = \perp, \end{cases}\Big)\\
[\![\mathsf{event}\ ev(\vec{D}); D']\!]\rho\,\kappa &= [\![D_1]\!]\rho\,\big(u_1 \mapsto \ldots\\
&\qquad \ldots [\![D_n]\!]\rho\,\big(u_n \mapsto\\
&\qquad \begin{cases} \mathsf{ev}(ev(\vec{u})).[\![D']\!]\rho\,\kappa & \vec{u} \in \mathcal{M}^n,\\ \kappa(\perp) & \text{else} \end{cases}\big)\ldots\big)
\end{aligned}$$

where $h(u_1, \ldots, u_n)$ is $f^{\uparrow}(\vec{u})$ for a constructor f and the function of Definition 3.1 for a destructor

g. Processes are translated by

$$
\begin{aligned}
[\![\mathbf{0}]\!]\rho &= \mathbf{0},\\
[\![\mathsf{out}(D, D'); P]\!]\rho &= [\![D]\!]\rho\left(u \mapsto [\![D']\!]\rho\left(v \mapsto \mathsf{out}(u, v).[\![P]\!]\rho \text{ if } u, v \in \mathcal{M}, \text{ else } \mathbf{0}\right)\right),\\
[\![\mathsf{in}(D, x); P]\!]\rho &= [\![D]\!]\rho\left(u \mapsto \begin{cases} \mathsf{in}(u).\big(m \mapsto [\![P]\!]\rho[x \mapsto m]\big) & u \in \mathcal{M},\\ \mathbf{0} & u = \bot, \end{cases}\right)\\
[\![P \mid Q]\!]\rho &= [\![P]\!]\rho \mid [\![Q]\!]\rho,\\
[\![!P]\!]\rho &= ![\![P]\!]\rho,\\
[\![\mathsf{new}\ a; P]\!]\rho &= \mathsf{new}.\big(m \mapsto [\![P]\!]\rho[a \mapsto m]\big),\\
[\![\mathsf{let}\ x = D\ \mathsf{in}\ P\ \mathsf{else}\ Q]\!]\rho &= [\![D]\!]\rho\left(u \mapsto \begin{cases} [\![P]\!]\rho[x \mapsto u] & u \in \mathcal{M},\\ [\![Q]\!]\rho & u = \bot, \end{cases}\right)\\
[\![\mathsf{if}\ D\ \mathsf{then}\ P\ \mathsf{else}\ Q]\!]\rho &= [\![D]\!]\rho\left(u \mapsto \begin{cases} [\![P]\!]\rho & u = \mathsf{true},\\ [\![Q]\!]\rho & u \in \mathcal{M} \setminus \{\mathsf{true}\},\\ \mathbf{0} & u = \bot, \end{cases}\right)\\
[\![\mathsf{event}\ ev(\vec{D}); P]\!]\rho &= [\![D_1]\!]\rho\left(u_1 \mapsto \ldots\right.\\
&\qquad \left.\ldots [\![D_n]\!]\rho\left(u_n \mapsto \begin{cases} \mathsf{ev}(ev(\vec{u})).[\![P]\!]\rho & \vec{u} \in \mathcal{M}^n,\\ \mathbf{0} & \text{else} \end{cases}\right)\ldots\right).
\end{aligned}
$$

The process $\mathbf{0}$ stands where the semantics of Section 2 has no transition. The translation of P_0 is $[\![P_0]\!]\rho_0$. A translated process mentions no ν_k except through new.

Remark 3.3. In the development an expression is a program in the continuation monad with answer type Proc returning a may-fail message, and a process a program in the same monad run by $\mathrm{run}(b) = b(\lambda_.\,\mathbf{0})$, with primitives $\mathrm{send}(c, m) = \lambda\kappa.\,\mathsf{out}(c, m).\kappa()$, $\mathrm{recv}(c) = \lambda\kappa.\,\mathsf{in}(c).\kappa$, $\mathrm{fresh} = \lambda\kappa.\,\mathsf{new}.\kappa$, $\mathrm{emit}(m) = \lambda\kappa.\,\mathsf{ev}(m).\kappa()$, $\mathrm{fork} = \lambda\kappa.\,\kappa(\mathrm{tt}) \mid \kappa(\mathrm{ff})$, $\mathrm{repl} = \lambda\kappa.\,!\kappa()$ and $\mathrm{stop} = \lambda_.\,\mathbf{0}$; let and if are case analysis in the metalanguage. The clauses above are the unfolding of that presentation, and the program the translator emits from a ProVerif model is that presentation.

Definition 3.4 (Translation of queries)**.** For a valuation $\sigma : X \to \mathcal{M}$ write $\sigma M \in \mathcal{M}$ for the evaluation of M with free names as constants, and $\sigma\, ev(M_1, \ldots, M_n) = ev(\sigma M_1, \ldots, \sigma M_n) \in \mathcal{M}$ for an event. The *instance* of a query fact under σ is the query fact of Definition 1.19 given by $\sigma\,\mathsf{event}(e) = \sigma\,\mathsf{inj\text{-}event}(e) = \mathsf{event}(\sigma e)$, $\sigma\,\mathsf{attacker}(M) = \mathsf{knows}(\sigma M)$, or $\mathsf{learns}(\sigma M)$ when the fact is injective, $\sigma\,\mathsf{mess}(N, M) = \mathsf{mess}(\sigma N, \sigma M)$, $\sigma(M = N) = (\sigma M = \sigma N)$, $\sigma(M <> N) = (\sigma M \neq \sigma N)$ and $\sigma(F\ \mathsf{before}\ e) = \sigma F\ \mathsf{before}\ \sigma e$, an $\mathsf{inj\text{-}event}$ keeping its mark. A query q of Definition 2.5 is thus a query $\hat{q}$ over $(X \to \mathcal{M}) \to \mathrm{QFact}$ in the sense of Definition 1.20, each fact F becoming $\sigma \mapsto \sigma F$, and it denotes $\mathrm{Satisfies}([\![P_0]\!]\rho_0, \hat{q})$. By Proposition 1.21, $[\![P_0]\!]\rho_0$ being prefix-closed, $\mathsf{query}\ F \implies H$ with one fact on each side, neither injective, denotes $\mathrm{Corr}([\![P_0]\!]\rho_0,\ \sigma \mapsto \sigma F,\ \sigma \mapsto \sigma H)$, the survey's Definition 3.5 with $=$ read as $=_E$; $\mathsf{query}\ F$ denotes $\forall\sigma.\ \forall t \in \mathrm{Attacks}([\![P_0]\!]\rho_0).\ t \not\models \sigma F$; and $\mathsf{query}\ \mathsf{attacker}(M)$ with M closed denotes $\mathrm{Secret}([\![P_0]\!]\rho_0, \rho_0 M)$.

3.3 The correspondence

Names of the operational semantics are the free names together with the created names; terms are interpreted in $\mathcal{M}$ through a bijection ι between the names outside $\mathcal{N}_{\text{pub}} \cup \mathcal{N}_{\text{priv}}$, those that new may create, and $\mathbb{N}$, extended by $\iota a = a$ for free names, $\iota a' = \nu_{\iota a'}$ for created names, $\iota f(\vec{M}) = f(\iota\vec{M})$, $\iota\, ev(\vec{M}) = ev(\iota\vec{M})$ for an event and $\iota\,\mathsf{fail} = \bot$; $\iota M = \iota M'$ exactly when $M =_E M'$, and likewise for events. Let ρ_ι be the environment with $\rho_\iota(a) = \iota a$ for every free or created name a.

Lemma 3.5 (Evaluation). *Let D be a closed expression over free and created names without* new, if, let *and* event. *Then $\mathcal{N}, D \to_t^* \mathcal{N}, U$ for some may-fail term U, and for every such U, $[\![D]\!]\rho_\iota = \kappa \mapsto \kappa(\iota U)$.*

Definition 3.6 (Labelled semantics). A *labelled term configuration* is (U, D) and a *labelled configuration* is $(U, \mathcal{P})$, with $U \subseteq \mathbb{N}$ finite, D a closed expression and $\mathcal{P}$ a finite multiset of closed processes, whose names are free names and symbols ν_k. The transitions of Section 2 become labelled steps, with $=_E$ read on $\mathcal{M}$: $\mathsf{new}\ a$ creates ν_k for any $k \notin U$ and adds k to U, with the label $\mathsf{new}(k)$; the steps on $\mathsf{event}\ e$ have the label $\mathsf{ev}([e])$, the event read as a message; the communication has the label $\mathsf{comm}([N], [M])$; the context rules carry the label of the step they lift; and every other step is a τ-step. There are besides the *offers*

$$(U, \mathcal{P} \cup \{\mathsf{out}(N, M); P\}) \xrightarrow{\mathsf{out}([N],[M])} (U, \mathcal{P} \cup \{P\}),$$
$$(U, \mathcal{P} \cup \{\mathsf{in}(N, x); P\}) \xrightarrow{\mathsf{in}([N],[M])} (U, \mathcal{P} \cup \{P\{M/x\}\}) \quad \text{for every closed term } M.$$

The communication is visible rather than a τ-step because a trace records a completed communication, which is what known reads the attacker's knowledge off. With $\overset{\alpha}{\Longrightarrow} = \overset{\tau}{\to}^* \overset{\alpha}{\to} \overset{\tau}{\to}^*$, the set of *weak traces* is

$$\mathrm{Tr}(U, \mathcal{P}) = \{\alpha_1 \cdots \alpha_n \in \mathrm{Act}^* \mid (U, \mathcal{P}) \overset{\alpha_1}{\Longrightarrow} \cdots \overset{\alpha_n}{\Longrightarrow}\},$$

and $(U, D) \overset{t}{\Rightarrow} (U', D')$ means that the term configuration (U, D) reaches (U', D') by steps whose visible labels, in order, are t.

Lemma 3.7 (Expressions). *Let D be a closed expression whose names are free names and symbols ν_k, let $U \subseteq \mathbb{N}$ be finite, let $\kappa : \mathcal{M}_\bot \to \mathrm{Proc}$ satisfy $\varepsilon \in \kappa(u)$ for every u, and let $t \in \mathrm{Act}^*$ have no letter of U in* $\mathrm{nonces}(t)$. *Then $t \in [\![D]\!]\rho\,\kappa$, where ρ extends ρ_0 by $\rho(\nu_k) = \nu_k$, if and only if $t = t_1 t_2$ for some $(U, D) \overset{t_1}{\Longrightarrow} (U', D')$ such that either $t_2 = \varepsilon$, or D' is a may-fail term, $t_2 \in \kappa(\iota D')$ and no letter of U' is in* $\mathrm{nonces}(t_2)$.

Theorem 3.8 (Adequacy). *For every closed process P whose names are free names and symbols ν_k, $[\![P]\!]\rho = \mathrm{Tr}(\emptyset, \{P\})$, where ρ extends ρ_0 by $\rho(\nu_k) = \nu_k$. For all finite multisets $\mathcal{P}, \mathcal{Q}$ of such processes,* $\mathrm{Tr}(\emptyset, \mathcal{P} \uplus \mathcal{Q}) = \mathrm{Tr}(\emptyset, \mathcal{P}) \mid \mathrm{Tr}(\emptyset, \mathcal{Q})$.

Let $\mathcal{A}_0 = \mathrm{Attacks}([\![P_0]\!]\rho_0)$. A trace Tr from $\mathcal{C}_Q$ is, through ι, a run of the labelled configuration $(\emptyset, \{P_0, Q\})$ none of whose labels is an offer; its *word* $t(\mathrm{Tr}) \in \mathrm{Act}^*$ is the sequence of its labels, which records the names it creates, its communications and its events.

Theorem 3.9 (Attacks). *(i) The words of the traces from $\mathcal{C}_Q$, over all $\mathcal{N}_{\text{pub}}$-adversaries Q, are exactly the traces in $\mathcal{A}_0$.*

(ii) For every trace Tr *from* $\mathcal{C}_Q$ *and closed term* M*, the attacker knows* M *after* Tr *if and only if* $\iota M \in \mathrm{known}(\langle\emptyset\rangle, t(\mathrm{Tr}))$.

Theorem 3.10 (Secrecy)**.** *Let* M *be a closed term with* $\mathrm{fn}(M) \subseteq \mathcal{N}_{\mathrm{pub}} \cup \mathcal{N}_{\mathrm{priv}}$ *and let* $s = \rho_0 M$*. Then* $\mathrm{Secret}([\![P_0]\!]\rho_0, s)$ *holds if and only if* P_0 *preserves the secrecy of* M*.*

Theorem 3.11 (Correspondences)**.** *Let* q *be a query. Then* P_0 *satisfies* q *against* $\mathcal{N}_{\mathrm{pub}}$*-adversaries if and only if* $\mathrm{Satisfies}([\![P_0]\!]\rho_0, \hat{q})$*, the property that* q *denotes by Definition 3.4.*

References

[1] Bruno Blanchet. Modeling and verifying security protocols with the applied pi calculus and ProVerif. *Foundations and Trends in Privacy and Security*, 1(1–2):1–135, October 2016. `../Reference/BlanchetFnTPS16.pdf`, doi:10.1561/3300000004.

[2] Bruno Blanchet, Ben Smyth, Vincent Cheval, and Marc Sylvestre. ProVerif 2.05: Automatic cryptographic protocol verifier, user manual and tutorial. October 2023. `../Reference/proverif-manual.pdf`.